

Accelerating TFHE: Scalable Multiplication Algorithms and Efficient Carry Handling

Rostin Shokri and Nektarios Georgios Tsoutsos

University of Delaware
{rostinsh, tsoutsos}@udel.edu

Abstract. Fully Homomorphic Encryption (FHE) enables private data processing on untrusted servers. However, FHE performance remains a critical bottleneck for applications such as machine learning, which rely heavily on non-linear operations, such as comparisons, and numerous ciphertext–ciphertext (CxC) and ciphertext–plaintext (CxP) multiplications. While modern FHE schemes such as TFHE efficiently handle non-linear operations, multiplication remains a significant performance bottleneck. This paper introduces new algorithms for CxC and CxP multiplication, as well as a ciphertext–plaintext dot-product algorithm, a critical kernel in machine-learning inference and convolution. Our approach uses Circuit Bootstrapping and Vertical Packing to generate partial products for CxC and CxP multiplication. The dot-product algorithm instead accumulates contributions from many multiplications directly, avoiding the cost of computing and adding each product separately. All three algorithms organize terms by output bit position, reduce them in parallel using carry-save compression or bound-driven reduction trees, and defer carry propagation to an efficient CMux-based final stage. We evaluate our algorithms and their parallel implementations against the default implementation in TFHE-rs and recent state-of-the-art approaches. Our results demonstrate speedups of up to $31.65\times$ on CPUs and $31.22\times$ on GPUs relative to TFHE-rs, and accelerate real-world neural network layers by up to $4.81\times$ over the fastest prior methods.

Keywords: Circuit Bootstrapping · Vertical Packing · Fully Homomorphic Multiplication · TFHE.

1 Introduction

Data-intensive applications, from large-scale genomic processing [5, 19, 22] to advanced machine learning workloads [18, 23, 29], are increasingly migrated to the cloud. This shift to remote data processing creates a paramount concern for data confidentiality [37], as it exposes sensitive information to new risks, such as untrusted servers reading and misusing personal data, as well as side-channel attacks that can extract private data from co-located tenants [39, 40]. While standard encryption like AES [12] protects data at rest and in transit, it is insufficient for end-to-end private computation, as data must be decrypted server-side before it can be processed, creating a window of vulnerability.

An effective solution is Fully Homomorphic Encryption (FHE) [17], which enables direct computation on encrypted data, ensuring that the secret key is never exposed to the server and that ciphertexts reveal nothing about the underlying plaintext. There are several FHE schemes [3, 7, 16] to choose from that come with different features and benefits. The TFHE (CGGI) scheme [9] in particular is effective for handling non-linear operations, which are critical for applications like machine learning [26, 33]. This capability is powered by its programmable bootstrapping (PBS) primitive, which can evaluate univariate functions on small, encrypted integers.¹

Despite TFHE’s strength in non-linear operations, its performance for arithmetic operations, specifically ciphertext–ciphertext (CxC) as well as ciphertext–plaintext (CxP) multiplication, remains a critical bottleneck. These operations are the dominant cost in many privacy-preserving workloads [13, 27], including the convolution and linear layers of neural networks [26, 33]. The original PBS paradigm, which operates on small bit widths, is inefficient for the large bit width multiplications (e.g., 16-bit or 32-bit) required by these workloads.

Circuit bootstrapping (CBS) and vertical packing (VP) were introduced as part of TFHE’s leveled-evaluation mode [8]. Recent refinements substantially accelerate CBS [35, 36], enabling practical evaluation of larger bit width lookup tables via VP and applications such as AES transciphering [35]. Although CBS remained the principal bottleneck in this mode, these optimizations have reduced its latency substantially, making the computational model practical.

Hybrid pipelines that evaluate linear layers in CKKS and non-linear ones in TFHE are an established design for approximate workloads, supported by dedicated switching procedures [24] and by compilers that place the switches automatically [30]. They are complementary to this work rather than an alternative to it. Switching from TFHE to CKKS incurs substantial computational cost and requires CKKS bootstrapping. Consequently, hybrid pipelines are most effective when computations remain in CKKS for sufficiently long regions, whereas frequent transitions can outweigh the benefits of switching schemes.

This work develops a complete, composable framework for homomorphic integer arithmetic by integrating modern CBS and VP techniques. We reformulate homomorphic multiplication through a weighted-column representation that unifies partial-product generation, column reduction, carry resolution, and noise refreshing while producing outputs in a composable ciphertext encoding. This construction combines a closed two-bit ciphertext format, a public-bound column representation, two complementary reduction methods, and one fused carry terminal. Our main contributions are as follows:

- We introduce a weighted-column representation with public bounds. Binary columns use carry-save compressors that cost one blind rotation each, while larger magnitudes use parallel sum-then-split rounds.

¹ The blind-rotation accumulator is negacyclic, so a PBS directly evaluates only functions satisfying $f(x + N) = -f(x)$. Reserving one padding bit confines the plaintext to half the domain, where any univariate function is representable; we use that convention throughout.

- We give algorithms for ciphertext–plaintext multiplication (CxP), ciphertext–ciphertext multiplication (CxC), and the ciphertext–plaintext dot product (CxP-DP). CxP uses shared LUTs over 16-bit windows, CxC uses indexed VP for encrypted 8-by-8-bit limbs, and CxP-DP forms its columns using only LWE additions selected by public scalar bits.
- We fuse the final carry computation into an encrypted CMux carry-select stage that writes two output bits per GLWE coefficient. Across the three algorithms, our framework achieves speedups of $1.18\text{--}31.65\times$ on a 64-thread CPU and $1.59\text{--}31.22\times$ on a single GPU relative to TFHE-rs, and accelerates AlexNet layer multi-scalar multiplications by $4.47\text{--}4.81\times$ over the fastest prior method.

Roadmap: Section 2 reviews the required TFHE primitives. Section 3 defines CC2, the shared column method, and our three proposed multiplication algorithms. Section 4 details the evaluation protocol and cryptographic parameters, reports the performance of CxP, CxC, and CxP-DP, and compares CPU and GPU implementations. Section 5 compares our framework with prior work, and Section 6 concludes the paper.

2 Background

2.1 Fully Homomorphic Encryption (FHE)

Fully Homomorphic Encryption (FHE) evaluates arbitrary computations directly on encrypted data, so sensitive values are never decrypted during processing. Modern schemes inject noise that accumulates with each homomorphic operation until decryption fails; *bootstrapping* homomorphically evaluates the decryption function to reset it and permit further computation. Security rests on Learning with Errors (LWE) [28] and its ring variant (RLWE) [25], both believed post-quantum secure, and homomorphic addition and multiplication together suffice to express any computation.

Word-wise schemes such as BFV [16] and BGV [3] “batch” many integers into a single RLWE ciphertext and support addition and multiplication on the packed data; CKKS [7] batches likewise but targets approximate real arithmetic. Bit-wise schemes encrypt each bit separately: FHEW [15] uses one LWE ciphertext per bit and *gate bootstrapping* to evaluate a Boolean gate and reset noise simultaneously. TFHE [9] generalizes this with *programmable bootstrapping* (PBS), whose fast blind rotation, combined with binary secret keys, evaluates a univariate function as part of the refresh. A PBS completes in tens of milliseconds on a CPU [9], and lets TFHE handle small integers of one to four bits with far smaller lattice parameters than word-wise schemes require.

2.2 TFHE Primitives

TFHE’s PBS-oriented evaluation mode relies on PBS, key switching, and modulus switching [9]. Chillotti et al. subsequently introduced a leveled-evaluation

mode based on circuit bootstrapping and vertical packing [8]; later work, including Bergerat et al., applied and optimized these primitives for larger-precision integer inputs [2]. These core operations are described next.

Programmable Bootstrapping (PBS). A PBS simultaneously refreshes a noisy LWE ciphertext and applies a univariate function f to its encrypted message μ_{in} [9, Alg. 9]. **ModSwitch** first maps the input modulus from q down to $2N$, where N is the GLWE ring dimension. **BlindRotate** then rotates a GLWE accumulator encoding the lookup table of f , using the homomorphic multiplexer (CMux) and its underlying external product. **SampleExtract** finally takes the constant coefficient of the rotated accumulator, yielding an LWE ciphertext under the GLWE key S that encrypts $f(\mu_{in})$. The input is an LWE ciphertext under the key s of dimension n_{LWE} , whereas the extracted output is under the coefficient embedding of S , of dimension kN ; a key switch of base $\mathcal{B}_{ks}$ and level ℓ_{ks} returns it to s , which is what makes successive bootstraps composable.

Circuit Bootstrapping (CBS). CBS lifts an LWE ciphertext into a GGSW ciphertext holding a single message bit. A GGSW of decomposition depth ℓ and base β comprises $(k+1)\ell$ GLWE ciphertexts under the k -component GLWE key $S = (S_1, \dots, S_k)$, of the form $\text{GLWE}_S(m \cdot S_i \cdot \frac{q}{\beta^j})$ for $1 \leq j \leq \ell$ and $0 \leq i \leq k$. Here $(S_0, \dots, S_k)$ is not the key but the extended gadget-secret vector, with the convention $S_0 = -1$: the $i \geq 1$ blocks encrypt the key-dependent products $m S_i$, and the $i = 0$ block encrypts $-m$, the row that the external product pairs with the body of the multiplicand ciphertext. Recent pipelines [35,36] split CBS into a refresh phase and a conversion phase. The **PBSmanyLUT** refresh omits sample extraction, so one blind rotation returns, for a parameter ϑ , 2^ϑ evenly spaced coefficients of the rotated accumulator, one per packed lookup table; here these are 2^ϑ scaled encryptions of the same bit, and when 2^ϑ is at least the GGSW level count $\ell_{ggs w}$ one blind rotation supplies every GLev level of a selector, so a CBS costs one blind rotation plus its conversion. The conversion applies a **Homomorphic Trace (HomTrace)**, a GLWE key switch that isolates the message in monomial terms while zeroing the others, followed by **Scheme Switching (SS)**, an external product with a public scheme-switching evaluation key, itself a set of GGSW encryptions of the secret-key polynomials, which reconstructs the key-dependent blocks of a valid GGSW; the server never handles the secret key itself. A *batched* CBS refreshes an LWE ciphertext carrying τ message bits once and converts it into τ separate one-bit selectors, so τ selectors cost one blind rotation rather than τ ; binary CBS is the case $\tau = 1$. Amortizing one blind rotation over several outputs in this way follows the multi-value bootstrapping technique of Carpov et al., which evaluates several lookup tables from a single blind rotation by factoring their test polynomials [4]. These outputs are not divided between the bits: each of the 2^ϑ outputs carries one GLev level, and every per-level accumulator still depends on the whole τ -bit value. The conversion separates the bits per level, isolating the most significant bit directly and each remaining bit in a complemented basis, $(b_1, b_0 \oplus b_1)$ at $\tau = 2$; the VP lookup

tables are re-indexed into the same basis at no cost. All conversions in this work use *RevHomTrace* of Lee and Yoon [20], which modulus-switches from q to $q/2$ before each trace automorphism and evaluates them in reverse order. This removes the phase-amplification problem and bounds the trace-noise variance by $O(N \log N)$, at essentially the same asymptotic cost as the standard HomTrace.

2.3 Vertical Packing (VP)

Vertical packing (VP) evaluates a wider lookup table by using several GGSW-encrypted bits together as the table index [8]. Rather than bootstrapping each small operation separately, VP folds the selection logic into one procedure and acts as a homomorphic table lookup: the encrypted index chooses a row, and the server cannot infer which row was selected. For w input bits and ring dimension N , the cost depends on w relative to $\log_2 N$. When $w \leq \log_2 N$ the evaluation is a single blind rotation of the accumulator, costing w , and hence at most $\log_2 N$, CMux operations. This blind rotation differs from the one inside a PBS: it runs over the w GGSW selectors rather than over the n_{LWE} bootstrapping-key entries, so it is far cheaper, since $n_{\text{LWE}} \gg \log_2 N$ at typical parameters; throughout this paper, blind-rotation counts refer to PBS blind rotations only, and VP is counted as CMux work. When $w > \log_2 N$, a CMux tree must first reduce the inputs, adding $2^{w - \log_2 N} - 1$ operations and growing exponentially in the excess. On the output side, one VP evaluation yields an LWE ciphertext holding $\log_2 p$ bits for a power-of-two message modulus p , so a full w -to- w bit table needs $\lceil w / \log_2 p \rceil$ evaluations; an 8-to-8 bit table with $p = 2$ therefore takes eight VP evaluations, which we call one *VP family*.

2.4 Threat Model

We consider the standard outsourced-computation setting for FHE: a client encrypts inputs under their secret key and sends the ciphertexts and public evaluation keys to an untrusted server. The baseline adversary is honest-but-curious, observing public keys, ciphertexts, and intermediate encrypted states, but not the secret key. For applications that expose decryption results we additionally account for IND-CPA^D security, introduced by Li and Micciancio for approximate FHE, which grants the adversary restricted decryption access for ciphertexts whose plaintext it should already know [21]. Cheon et al. subsequently showed that exact FHE implementations with imperfect correctness also admit distinguishing and key-recovery attacks in this model, including a simulated attack against TFHE-rs [6]. We therefore treat the correctness-failure probability as security-relevant, and every parameter set we construct targets $P_{\text{fail}} \leq 2^{-128}$; for related work we use the parameter set published with each method, together with the failure probability that parameter set targets. We do not address malicious-server verifiability, access-pattern leakage, or implementation-level side channels such as timing, cache, or cross-tenant attacks.

3 Proposed Construction

In this section, we present three novel algorithms for ciphertext–plaintext multiplication, ciphertext–ciphertext multiplication, and ciphertext–plaintext dot products.

Throughout this section, operands and results are n -bit integers modulo 2^n , where n is a multiple of eight. Our algorithms support both signed and unsigned integers.

All three algorithms share the same input and output encoding. An n -bit integer is carried as $n/2$ LWE ciphertexts, each holding two adjacent message bits together with spare carry space and a padding bit; we call this the closed CC2 format, and Section 3.1 fixes it precisely. Because every algorithm both consumes and produces CC2, the three operations compose without any conversion between them.

From that common encoding the flow is the same in each case. A front end turns the operands into *weighted columns*, a list of encrypted terms each tagged with a public output position and a public upper bound on its value. CxC and CxP build those terms by lifting the encrypted operand bits to GGSW selectors with circuit bootstrapping and evaluating product lookup tables with vertical packing; CxP-DP builds them with plain LWE additions routed by the public scalar bits, and needs neither. A shared back end then reduces every column until it holds at most three, resolves the carry chain once, and emits fresh CC2 chunks. We present CxC first because it exercises every stage, then obtain CxP by replacing one encrypted operand with a public scalar, and finally give the VP-free CxP-DP route.

3.1 Ciphertext–Ciphertext Multiplication

The closed CC2 format. Let $q = 2^{64}$ be the native torus modulus. For $X = \sum_{j=0}^{n-1} x_j 2^j$, CC2 stores two adjacent bits in one LWE ciphertext:

$$C_k = \text{Enc}_{\text{LWE}}((x_{2k} + 2x_{2k+1})\Delta_{\text{CC2}}), \quad \Delta_{\text{CC2}} = 2^{52}. \quad (1)$$

The scale follows from a 12-bit padded domain. We write $B_{\text{CC2}} = 11$ for the number of positions available below the padding bit: two message bits and nine carry bits, so $\Delta_{\text{CC2}} = 2^{63-B_{\text{CC2}}}$. At an operation boundary, C_k contains only $x_{2k} + 2x_{2k+1} \in \{0, 1, 2, 3\}$, and the carry positions are zero. During evaluation, the same plaintext interval carries any public-bounded value in $[0, 2^{11} - 1]$ through linear operations.

The nine carry positions are mainly used by CxP-DP. They allow many CC2 contributions to be added as LWE ciphertexts before a bootstrap is needed. A column is admissible only if its term bounds sum to less than 2^{11} . No lookup table is ever evaluated over this full 11-bit domain: at ring dimension N the modulus switch of a bootstrap retains $2N$ phase positions, of which only N fall in the padded half of the domain, too few to separate 2^{11} values. Instead, values in $[0, 2^{11} - 1]$ are only accumulated and rescaled linearly, and before every

bootstrap the ciphertext is multiplied by a public power of two so that the effective plaintext space of that bootstrap is at most two message bits plus the padding bit (Section 3.3 gives the rescaling rules). The extra padding bit is not carry space. It keeps every valid plaintext in one half of the negacyclic PBS domain, leaving a guard interval at the torus wraparound, which is what makes those public rescalings and the identity and bit-extraction reads below sound.

CxC and CxP lift a CC2 chunk to the no-padding two-bit CBS encoding by multiplying the LWE ciphertext by 2^{10} . This linear operation moves the scale from 2^{52} to 2^{62} and does not require a bootstrap. CxP-DP instead keeps the original 2^{52} scale so that its nine carry positions remain available.

As an interface, CC2 fixes both the key and the noise. Every chunk is an LWE ciphertext under the same key s of dimension n_{LWE} that the PBS consumes, so no key switch separates the three operations. Each operation accepts chunks whose variance is at most twice the PBS output variance and returns chunks meeting that same bound; the padded emission delivers one times that variance, and the fallback emission, two half-torus sign bootstraps per chunk of which the high read carries a doubled output scale, delivers two, so two is the contract producers are measured at and consumers may assume. A composition is admissible only if the term bounds of every accumulated column sum below $2^{B_{\text{CC2}}}$; full admissibility additionally requires the modeled failure budget of the failure analysis below.

CBS, VP, and weighted columns. A two-bit batched CBS applies one PBSmanyLUT blind rotation to a lifted CC2 chunk and produces the two GGSW selectors required by VP [35]. The conversion of each selector uses the reverse-normalized HomTrace described in Section 2, followed by scheme switching. In CxC, both n -bit operands contain $n/2$ chunks, so the input stage uses n batched-CBS blind rotations. The resulting selectors are created once and reused by every partial-product VP evaluation.

We organize all partial products as weighted columns. A term is an LWE ciphertext encrypting a value $a \in [0, b]$ for a public bound b , tagged with a column index c that assigns it weight 2^c . For column lists $\mathcal{C} = (\mathcal{C}_0, \dots, \mathcal{C}_{n-1})$, define

$$\Phi(\mathcal{C}) = \sum_{c=0}^{n-1} 2^c \sum_{(a,b) \in \mathcal{C}_c} a \pmod{2^n}. \quad (2)$$

Terms in columns $c \geq n$ are discarded because their weights vanish modulo 2^n . Every bound is computed from public dimensions and public scalars; it does not depend on an encrypted value.

For CxC, write $X = \sum_{i=0}^{d-1} X_i 2^{8i}$ and $Y = \sum_{j=0}^{d-1} Y_j 2^{8j}$, where $d = n/8$. For every pair $i + j < d$, the encrypted 8-bit limbs X_i and Y_j index one 16-input table family whose evaluations return the exact product, one surviving output bit per evaluation, in columns $8(i + j)$ through $\min(8(i + j) + 15, n - 1)$. Pairs with $i + j \geq d$ are omitted because all their bits have weight at least 2^n . Thus,

the leaf columns satisfy

$$XY \equiv \sum_{\substack{0 \leq i, j < d \\ i+j < d}} X_i Y_j 2^{8(i+j)} \pmod{2^n}. \quad (3)$$

The valid region contains $d(d+1)/2$ independent families of at most 16 evaluations each. The low eight columns contain one product bit each and can pass directly to the output. For $c \geq 8$, the raw column height is

$$h_c^C = 2\lfloor c/8 \rfloor + 1, \quad \max_c h_c^C = 2d - 1. \quad (4)$$

Reduction and the final carry. VP emits each product bit at the padded two-bit scale 2^{61} , as an LWE ciphertext under the coefficient embedding of the GLWE key S ; the key switch of Section 2 returns it to s before any later bootstrap, and the failure model charges that switch. Therefore, two or three bits in one column can be added without changing the encoding. A Dadda reduction [11] reduces the bit stacks to height at most three. For a half or full compressor, let u be the sum of its two or three input bits. One two-output PBSmanyLUT evaluates

$$\text{lo} = u \bmod 2, \quad \text{hi} = \lfloor u/2 \rfloor. \quad (5)$$

The parity ciphertext lo remains in the current column and the carry ciphertext hi moves to the next column. The two outputs share one blind rotation. All compressors in a Dadda layer are independent; only consecutive layers are dependent.

After reduction, the at most three bits left in column c are summed with plain LWE additions at their shared 2^{61} scale into one phase $A_c \in [0, 3]$. A bound-one column is read by one binary CBS; a bound-two or bound-three column is read by two, giving $A_c = 2\text{hi}_c + \text{lo}_c$ with lo_c the least significant bit and hi_c the most significant bit of the same LWE phase. Because $\sum_c 2^c A_c = \sum_c 2^c (\text{lo}_c + \text{hi}_{c-1})$, the high selector of column c is consumed as an input of column $c+1$ rather than as a carry, so every column presents at most two bits to the ripple. With $\text{hi}_{-1} = 0$ and $r_0 = 0$, the terminal computes

$$S_c = \text{lo}_c + \text{hi}_{c-1} + r_c, \quad (6)$$

$$y_c = S_c \bmod 2, \quad r_{c+1} = \lfloor S_c/2 \rfloor.$$

Here $S_c \leq 3$, so $r_{c+1} \in \{0, 1\}$ and the carry chain has only two carry states, which the terminal evaluates in carry-select form. One CMux applies lo_c and a second applies hi_{c-1} when column $c-1$ had bound at least two, so a column costs one CMux level, or two when its less significant neighbour was not already a single bit.

Concretely, the terminal keeps one GLWE carry state per incoming-carry value, two in total, each accumulating the encrypted output suffix under its carry hypothesis as in a carry-select adder, and walks the columns of the carry chain from most to least significant. For each state, the candidate successors are

resolved by CMux on those selectors, and y_c is added into GLWE coefficient $\lfloor c/2 \rfloor$ of the resolved state at scale $2^{61+(c \bmod 2)}$, so one sample extraction per coefficient yields a padded two-bit chunk directly. The safe CMux depth is the largest depth at which the total failure probability of the emission wave stays within the operation budget less one bit reserved for the remaining classes, found by binary search on the exact model. If one carry chain would exceed it, the public plan selects the smallest balanced CC2-aligned segmentation in which every segment's depth, plus one boundary CMux per segment after the first, respects that limit. A segment writes its outgoing carry into an unused GLWE coefficient beyond the outputs; one sample extraction and one binary CBS refresh it into the selector that chooses between the next segment's two precomputed states. Finally, one identity PBS per two-bit output chunk restores the CC2 scale, for an emission cost of $n/2$ blind rotations.

CxC algorithm. Algorithm 1 gives the complete construction. The notation $\text{Terminal}^{2\text{Bin},\text{pad}}_{\leq 3}$ includes public segmentation when it is required by the noise bound.

The algorithms use the following public operators. Lift is the 2^{10} scaling of Section 3.1 and CBS_2 the two-bit batched CBS above. IndexedVP maps two 8-bit selector groups to the surviving bits of their product, one LWE per bit; VP maps one 16-bit group and a LUT prefix to the same; PlaceBits appends those bits to the columns at a public offset with their bounds. DaddaReduce $_{\leq 3}$ applies Equation (12); PlanGroups and SumSplitTree $_{\leq 3}$ apply the grouping rule and Equation (11) of Section 3.3; $\text{Terminal}^{2\text{Bin},\text{pad}}_{\leq 3}$ applies Equation (6) with the segmentation rule above; and EmitCC2 is the closing identity PBS.

Algorithm 1 Ciphertext–ciphertext multiplication (CxC)

Input: $\text{CC2}(X), \text{CC2}(Y)$; n a multiple of 8

Output: $\text{CC2}(XY \bmod 2^n)$

- 1: $(G_X, G_Y) \leftarrow \text{CBS}_2(\text{Lift}(X) \parallel \text{Lift}(Y))$
 - 2: $\mathcal{C} \leftarrow (\emptyset, \dots, \emptyset)$
 - 3: **for all** (i, j) with $0 \leq i, j < n/8$ and $i + j < n/8$, **in parallel do**
 - 4: $P_{i,j} \leftarrow \text{IndexedVP}(G_{X,i}, G_{Y,j}, \text{mul})$
 - 5: $\text{PlaceBits}(\mathcal{C}, P_{i,j}, 8(i + j))$
 - 6: $\mathcal{C} \leftarrow \text{DaddaReduce}_{\leq 3}(\mathcal{C})$
 - 7: $Z \leftarrow \text{Terminal}^{2\text{Bin},\text{pad}}_{\leq 3}(\mathcal{C}, \text{prefix} = 8)$
 - 8: **return** $\text{EmitCC2}(Z)$
-

3.2 Ciphertext–Plaintext Multiplication

CxP follows the same construction as CxC after changing the leaf stage. Only one operand is encrypted, so only its $n/2$ CC2 chunks undergo two-bit batched CBS. The public scalar is embedded in the VP LUT rather than represented by GGSW selectors. This halves the input-CBS blind rotations and replaces the triangular

8-by-8 limb-product matrix with 16-bit windows of the encrypted operand. The weighted columns, Dadda reducer, two-binary terminal, segmentation rule, and CC2 emission are unchanged.

Let $s \in \mathbb{Z}_{2^n}$ be public and write $X = \sum_{t=0}^{T-1} X_t 2^{16t}$, where $T = \lceil n/16 \rceil$. The scalar-dependent LUT family is

$$F_j(w) = \text{bit}_j(sw), \quad 0 \leq w < 2^{16}. \quad (7)$$

Window t evaluates the prefix $F_0, \dots, F_{n-16t-1}$ and places its outputs starting at column $16t$. When n is not a multiple of 16, the last window carries only $n - 16(T-1)$ encrypted bits and is zero-extended to the 16-bit index; the padding bits are public zeros, so (7) is unchanged and the missing selectors are simply not created. Hence

$$sX \equiv \sum_{t=0}^{T-1} (sX_t) 2^{16t} \pmod{2^n}. \quad (8)$$

Column c initially contains $h_c^P = \lfloor c/16 \rfloor + 1$ bits. The first 16 columns have height one and bypass the terminal. When $n \leq 16$, there is one window, every column already contains its final bit, and the common plan has zero Dadda layers and an empty carry chain.

Algorithm 2 Ciphertext–plaintext multiplication (CxP)

Input: $\text{CC2}(X)$, public n -bit scalar s ; n a multiple of 8

Output: $\text{CC2}(sX \bmod 2^n)$

- 1: $G \leftarrow \text{CBS}_2(\text{Lift}(X))$
 - 2: $\mathcal{C} \leftarrow (\emptyset, \dots, \emptyset)$
 - 3: **for all** 16-bit windows t **in parallel do**
 - 4: $P_t \leftarrow \text{VP}(G_t, \{F_j\}_{j < n-16t})$
 - 5: $\text{PlaceBits}(\mathcal{C}, P_t, 16t)$
 - 6: $\mathcal{C} \leftarrow \text{DaddaReduce}_{\leq 3}(\mathcal{C})$
 - 7: $Z \leftarrow \text{Terminal}_{\leq 3}^{2\text{Bin}, \text{pad}}(\mathcal{C}, \text{prefix} = \min(16, n))$
 - 8: **return** $\text{EmitCC2}(Z)$
-

3.3 Ciphertext–Plaintext Dot Product

Given public scalars s_i and encrypted values $X^{(i)}$, CxP-DP computes the dot product

$$\sum_{i=0}^{m-1} s_i X^{(i)} \bmod 2^n.$$

Running CxP independently for every vector element and adding the m products would give each instance its own selectors, VP work, and scratch space, so GGSW and VP storage would grow linearly with m . That is unattractive when m is in the hundreds or thousands, and on a GPU it can exhaust memory before arithmetic throughput is reached.

CxP-DP therefore does not use VP and does not circuit-bootstrap its inputs. Let the k th CC2 chunk of $X^{(i)}$ encrypt

$$\hat{x}_k^{(i)} = x_{2k}^{(i)} + 2x_{2k+1}^{(i)} \in [0, 3].$$

Expanding the public scalar bits gives

$$s_i X^{(i)} \equiv \sum_{\substack{0 \leq v < n, \ 0 \leq k < n/2 \\ \text{bit}_v(s_i)=1, \ v+2k < n}} \hat{x}_k^{(i)} 2^{v+2k} \pmod{2^n}. \quad (9)$$

Thus, a set scalar bit only copies an existing LWE chunk into a publicly known column. The copies are combined with LWE additions, which need no evaluation key and no bootstrap. The nine carry positions of Section 3.1 provide exactly this accumulation headroom.

For a public group g of vector indices, the initial bound of column c is

$$U_{g,c} = 3 \sum_{i \in g} \sum_{\substack{v+2k=c \\ 0 \leq v < n, \ 0 \leq k < n/2}} \text{bit}_v(s_i). \quad (10)$$

The planner is deterministic: it enumerates the admissible group sizes, simulates each candidate's complete reduction schedule on the public bounds, and keeps the cheapest schedule that fits the CC2 domain and the failure budget. A single flat merge is preferred whenever it meets that budget. Otherwise, each group is normalized separately to column bounds at most three and the summaries are merged pairwise through a balanced binary hierarchy, an unpaired summary passing unchanged to the next level. Every merge node is reduced back to bound three before it enters the next level, so workloads larger than one group's carry capacity use additional public reduction rounds rather than a larger ciphertext format.

CxP-DP reduces bounded magnitudes rather than stacks of binary product bits. Suppose a column contains $A \in [0, U]$ at weight 2^c . Let $L = \lceil \log_2(U+1) \rceil$ and $\alpha = \min(L, n-c)$. If all L bits survive modulo 2^n , the reducer preserves the top one or two bits as a refreshed small value and extracts the other bits with sign PBS calls. If the value is truncated at bit n , it extracts only the surviving bits. In either case, the emitted terms satisfy

$$\begin{aligned} 2^c A &\equiv \sum_{j=0}^{r-1} 2^{c+j} \text{bit}_j(A) \\ &\quad + \mathbf{1}_{\nu > 0} 2^{c+r} \lfloor A/2^r \rfloor \pmod{2^n}, \end{aligned} \quad (11)$$

where $\nu = 0$ when $\alpha < L$, since the high part then vanishes modulo 2^n , and $\nu = \min(\alpha, 2)$ when $\alpha = L$, so one identity PBS refreshes the retained high part instead of extracting it bit by bit; $r = \alpha - \nu$ is the number of extracted low bits. A column with $U \leq 1$ already holds a single bit, so $\nu = r = 0$ and no split is issued. Columns are processed in parallel, and rounds continue until every final

Algorithm 3 Ciphertext–plaintext dot product (CxP-DP)

Input: $\{\text{CC2}(X^{(i)}), s_i\}_{i=0}^{m-1}$; n a multiple of 8

Output: $\text{CC2}(\sum_i s_i X^{(i)} \bmod 2^n)$

- 1: $\mathcal{G} \leftarrow \text{PlanGroups}(\{s_i\}, n, B_{\text{CC2}} = 11)$
 - 2: **for all** $g \in \mathcal{G}$ and $0 \leq c < n$ **in parallel do**
 - 3: $\mathcal{I}_{g,c} \leftarrow \{(i, v, k) : i \in g, 0 \leq v < n, 0 \leq k < n/2, v + 2k = c, \text{bit}_v(s_i) = 1\}$
 - 4: $A_{g,c} \leftarrow \sum_{(i,v,k) \in \mathcal{I}_{g,c}} X_k^{(i)}, \quad U_{g,c} \leftarrow 3|\mathcal{I}_{g,c}|$
 - 5: $\mathcal{C} \leftarrow \text{SumSplitTree}_{\leq 3}(\{(A_{g,c}, U_{g,c})\}_{g,c})$
 - 6: $Z \leftarrow \text{Terminal}_{\leq 3}^{\text{2Bin, pad}}(\mathcal{C}, \text{prefix} = 0)$
 - 7: **return** $\text{EmitCC2}(Z)$
-

bound is at most three. The same two-binary padded terminal used by CxC and CxP then resolves the one remaining carry chain.

The extraction mechanics are as follows. A *sign PBS* is a bootstrap whose test polynomial takes only the two values $\pm \Delta_{\text{CC2}}/2$ on the two halves of the torus; adding the public constant $\Delta_{\text{CC2}}/2$ to its output yields the read bit at the column scale. The two possible phases are 0 and $q/2$, so the noise margin is a full quarter of the torus and the read needs no padding bit. Bits are extracted least significant first. To read $\text{bit}_j(A)$, the column ciphertext, from which bits $0, \dots, j-1$ have already been subtracted, is multiplied by the public constant $2^{B_{\text{CC2}}-j}$: bit j moves to the half-torus position and every higher bit wraps off the torus. The fresh bit, rescaled by 2^j , is then subtracted before the next read, so the r extractions within one column form a sequential chain, while distinct columns still run in parallel. After the r subtractions the remaining phase encodes exactly $\lfloor A/2^r \rfloor \cdot 2^r$ at the column scale; multiplying it by $2^{B_{\text{CC2}}-r-\nu}$ aligns $\lfloor A/2^r \rfloor$ to a padded ν -bit domain, where the identity PBS of Equation (11) refreshes it. Every lookup table in the split therefore acts on at most two message bits, never on the full B_{CC2} -bit interval.

3.4 Cost Analysis

Every column bound, reduction round, compressor, terminal selector, and segment boundary is fixed before encrypted evaluation begins and depends only on public information: on n and the fixed limb or window geometry for CxC and CxP, and additionally on m , the public scalar bits, and the resulting group size for CxP-DP. The framework is therefore independent of the encrypted operands, and the counts below are exact rather than expected. Together with the target chain, grouping rule, and segmentation rule above, every quantity in Table 1 is computable from public data alone.

Dadda counts. Let $h_{\lambda,c}$ be the height of column c entering a Dadda layer with target height h_{λ}^* , and let $\kappa_{\lambda,c}$ be the carries entering that column. Starting with

$\kappa_{\lambda,0} = 0$, the public recurrence is

$$\begin{aligned} E_{\lambda,c} &= \max(h_{\lambda,c} + \kappa_{\lambda,c} - h_{\lambda}^*, 0), \\ F_{\lambda,c} &= \lfloor E_{\lambda,c}/2 \rfloor, \quad H_{\lambda,c} = E_{\lambda,c} \bmod 2, \\ h_{\lambda+1,c} &= h_{\lambda,c} - 2F_{\lambda,c} - H_{\lambda,c} + \kappa_{\lambda,c}, \\ \kappa_{\lambda,c+1} &= F_{\lambda,c} + H_{\lambda,c}. \end{aligned} \tag{12}$$

Here $F_{\lambda,c}$ and $H_{\lambda,c}$ are the full- and half-compressor counts. The target sequence is the classical Dadda chain: starting from three, targets grow by $h^* \mapsto \lfloor 3h^*/2 \rfloor$ while below the maximum initial height, and the layers apply the chain in descending order, ending at three. For $\text{op} \in \{\text{C}, \text{P}\}$, define

$$D_{\text{op}}(n) = \sum_{\lambda,c} (F_{\lambda,c}^{\text{op}} + H_{\lambda,c}^{\text{op}}). \tag{13}$$

The initial CxC heights are zero for the eight pass-through columns and $2\lfloor c/8 \rfloor + 1$ afterward. The initial CxP heights are $\lfloor c/16 \rfloor + 1$; its first $\min(16, n)$ columns are later passed through. Equation (12) therefore gives an exact width-dependent compressor count, not only an asymptotic bound.

Terminal and tree counts. For a final bound $b \leq 3$, let $\text{sel}(0) = 0$, $\text{sel}(1) = 1$, and $\text{sel}(2) = \text{sel}(3) = 2$. For each operation op , let $\eta_{\text{op}} = 1$ when at least one non-pass-through final bound is two or three, and let $\eta_{\text{op}} = 0$ otherwise. Define $Q_{\text{C}}(n)$ and $Q_{\text{P}}(n)$ as η_{op} times the sum of sel over their final non-pass-through columns. Define $Q_{\text{D}}(n, m, \mathbf{s})$ in the same way over the final CxP-DP tree bounds. This makes $Q_{\text{op}} = 0$ when every output column is already a bit and no terminal is needed. Let $S_{\text{C}}(n)$, $S_{\text{P}}(n)$, and $S_{\text{D}}(n, m, \mathbf{s})$ be their padded-terminal segment counts. By convention, $S_{\text{op}} = 1$ when no boundary refresh is needed, so every terminal has $S_{\text{op}} - 1$ boundary CBS operations. At the parameters of Table 2, segmentation activates only at $n = 512$, where $S_{\text{P}} = 2$ and $S_{\text{C}} = 3$; every smaller evaluated width has $S_{\text{op}} = 1$. For example, CxC at $n = 64$ has $D_{\text{C}} = 332$ and $Q_{\text{C}} = 112$, hence 540 blind rotations and 960 HomTrace operations in total.

For the CxP-DP tree, one decomposition of a bound $U > 1$ at column c costs

$$\begin{aligned} L &= \lceil \log_2(U + 1) \rceil, \quad \alpha = \min(L, n - c), \\ \rho(U, c) &= \begin{cases} \alpha, & \alpha < L, \\ \alpha - \min(\alpha, 2) + 1, & \alpha = L. \end{cases} \end{aligned} \tag{14}$$

Set $\rho(U, c) = 0$ for $U \leq 1$. The first case extracts every surviving bit because the value is truncated modulo 2^n . The second case extracts the low bits and refreshes the remaining one- or two-bit value with one identity PBS. Let $R_{\text{D}}(n, m, \mathbf{s})$ be the sum of $\rho(U, c)$ over every initial group, reduction round, and hierarchical merge in the public CxP-DP plan. Equation (10) makes this count exact for the public scalars. At fixed (n, m) it is necessarily scalar-dependent: all-one scalars give the per-group worst-case initial bound $U_{g,c} = 3\lceil g \rceil(\lfloor c/2 \rfloor + 1)$, whereas sparse scalars create fewer terms.

Table 1. Theoretical PBS and HomTrace counts by algorithm stage.

Operation	Stage	PBS / blind rotations	HomTrace
CxC	Two-operand $\tau = 2$ input CBS	n	$2\ell_{ggsw}n$
	$d(d+1)/2$ indexed VP families	0	0
	Dadada reduction	$D_C(n)$	0
	Terminal selectors and boundaries	$Q_C + S_C - 1$	$\ell_{ggsw}(Q_C + S_C - 1)$
	CC2 emission	$n/2$	0
	Total	$3n/2 + D_C + Q_C + S_C - 1$	$\ell_{ggsw}(2n + Q_C + S_C - 1)$
CxP	One-operand $\tau = 2$ input CBS	$n/2$	$\ell_{ggsw}n$
	T scalar-dependent VP families	0	0
	Dadada reduction	$D_P(n)$	0
	Terminal selectors and boundaries	$Q_P + S_P - 1$	$\ell_{ggsw}(Q_P + S_P - 1)$
	CC2 emission	$n/2$	0
	Total	$n + D_P + Q_P + S_P - 1$	$\ell_{ggsw}(n + Q_P + S_P - 1)$
CxP-DP	Public column formation	0	0
	Sum-split tree	$R_D(n, m, \mathbf{s})$	0
	Terminal selectors and boundaries	$Q_D + S_D - 1$	$\ell_{ggsw}(Q_D + S_D - 1)$
	CC2 emission	$n/2$	0
	Total	$R_D + Q_D + S_D - 1 + n/2$	$\ell_{ggsw}(Q_D + S_D - 1)$

Here $d = n/8$ and $T = \lceil n/16 \rceil$. One PBS denotes one blind rotation; a two-output Dadada compressor is therefore counted once. For the evaluated full-RevHomTrace path, one HomTrace entry denotes one GLWE trace. Each selector GGSW contains ℓ_{ggsw} GLev levels, so one CBS selector contributes ℓ_{ggsw} HomTrace operations. VP uses CMux and external products but no PBS or HomTrace.

The table separates total work from parallel depth. CxC and CxP execute their independent input CBS operations in one wave, their VP evaluations in parallel, and every Dadada layer as one compressor wave. Terminal selector CBS operations in a segment also form one wave; the carry-select CMux chain within that segment is sequential in the column direction. CxP-DP processes groups and columns in parallel, but successive sum-split rounds and hierarchy levels are dependent, as are the sign-extraction steps within one column decomposition. A segment boundary adds one further dependent binary CBS wave.

3.5 Correctness and Failure Analysis

Domain and noise checks. A public plan is accepted only if every pre-refresh sum stays inside its declared plaintext domain and its total modeled correctness-failure probability is at most 2^{-128} . For each class e of bootstraps or decodings, let μ_e be the distance from the expected phase to the nearest phase that yields an incorrect result, σ_e^2 its modeled variance, and K_e the number of such operations. We use the union bound

$$\gamma_e = \frac{\mu_e}{\sigma_e}, \quad p_e = \text{erfc}\left(\frac{\gamma_e}{\sqrt{2}}\right), \quad P_{\text{fail}} \leq \sum_e K_e p_e. \quad (15)$$

A failure is a PBS or CBS that returns an incorrect result, or an output chunk that decodes incorrectly. The classes are the bootstrap stages of Table 1 plus the

Table 2. Parameter sets for CxP, CxC, and CxP-DP. Our GPU implementation uses the Ours parameters with grouping-factor-four multi-bit blind rotation [10], as does TFHE-rs GPU.

Work	n_{LWE}	N	k	ℓ_{ks}	$\mathcal{B}_{ks}$	ℓ_{pbs}	$\mathcal{B}_{pbs}$	ℓ_{tr}	$\mathcal{B}_{tr}$	ℓ_{ss}	$\mathcal{B}_{ss}$	ℓ_{ggsw}	$\mathcal{B}_{ggsw}$	ϑ	τ	P_{fail}
Ours	768	1024	2	7	2^2	2	2^{15}	3	2^{12}	2	2^{17}	4	2^4	2	2	2^{-128}
tfhe-rs/SMOOTHIE [27, 38]	918	2048	1	4	2^4	1	2^{23}	—	—	—	—	—	—	—	—	$2^{-129.581}$
tfhe-rs GPU [38]	920	2048	1	5	2^3	1	2^{22}	—	—	—	—	—	—	—	—	$2^{-134.345}$
Head Start [13]	775	2048	1	7	2^2	1	2^{23}	—	—	—	—	—	—	1	—	2^{-64}
Scytale [32]	640	1024	2	7	2^2	2	2^{15}	5	2^9	2	2^{17}	2	2^7	1	2	$2^{-130.41}$

final output decoding. No other operation can fail on its own: reverse-normalized HomTrace and scheme-switch noise, VP and external products, LWE additions, and the CMux depth of a terminal segment only increase the variance σ_e of the next bootstrap or decoding that consumes the affected ciphertext. The union bound does not assume independent failures.

Following standard TFHE noise analysis, each accumulated error is modeled as a zero-mean Gaussian with the exactly propagated variance under the usual independence heuristic [2], so p_e is the tail of the modeled distribution rather than a distribution-free bound. We validated this model on encrypted test vectors: the measured variances at the CC2 contract points, covering the emission wave and the input lift, match the modeled variances to within 0.35 bits.

Correctness. We condition on every PBS and CBS returning its correct result and on every output chunk decoding to its correct value; the bound above covers the complement. Each front end’s displayed identity makes $\Phi(\mathcal{C})$ the required result, and compressors and splits preserve weighted value by construction. For the terminal, induction on c gives

$$\sum_{j=0}^{c-1} 2^j A_j = \sum_{j=0}^{c-1} 2^j y_j + 2^c (r_c + \text{hi}_{c-1}). \quad (16)$$

At $c = n$, the last term is zero modulo 2^n , so the emitted bits are the binary representation of the result and the closing identity PBS changes only their encoding and noise.

4 Experimental Evaluation

We implement our proposed algorithms in **tfhe-rs** v1.7.0 [38], a state-of-the-art TFHE library written in Rust. We adopt the refined CBS algorithm from [35], adapt it to this library, and use the new Centered Mean Noise Reduction from [14] to reduce modulus-switching errors, which allows us to generate more efficient parameters for the same P_{fail} .

Our parameter set and the TFHE-rs and SMOOTHIE baselines target failure probability at most 2^{-128} . The Head Start baseline uses the parameters published with that work, which target 2^{-64} ; we therefore identify it as a separate reference point rather than treating it as a failure-matched comparison. All

experiments are conducted on an Ubuntu workstation equipped with a 32-core AMD Threadripper 7970X CPU with two hardware threads per core (64 hardware threads total) and 160 GB of RAM. We estimate lattice security with the Lattice Estimator.² The estimator reports at least 128 bits of security for every parameter set.

Every timed interval ends with a carry-resolved ciphertext in that method’s documented output format, ours in closed CC2, so the complete emission wave is included; every algorithm’s public plan, including the CxP-DP group plan computed from the public scalars, is included in the reported times, while key generation and encryption lie outside every timer, and each timed result, computed on operands drawn from a fixed uniformly pseudorandom pattern, is decrypted and checked against the cleartext computation modulo 2^n . Baseline window sizes follow the settings reported by SMOOTHIE [27], with bucket doubling enabled for the multi-scalar baselines; Head Start publishes no windows of its own and reuses the SMOOTHIE settings, clamped to the window range its dot-product implementation supports [13].

4.1 Our Cryptographic Parameters

In terms of performance, the PBS and HomTrace operations are the most expensive. A GGSW ciphertext requires more memory than an LWE ciphertext, so limiting $\ell_{ggs w}$ is important. Our optimized parameters are presented in Table 2. Subject to the noise and correctness constraints of the complete pipeline, we choose $\ell_{ggs w} = 4$. CxP and CxC use batched CBS with $\tau = 2$ to read their operands. The CPU terminal of every proposed algorithm instead uses two binary CBS reads for each final column bounded by two or three.

For both **tfhe-rs** and [27] experiments, we use the TFHE-rs KS32 baseline parameter set with the Centered Mean Noise Reduction technique, shown alongside ours in Table 2. Our GPU baselines use the default TFHE-rs GPU multi-bit parameter set with grouping factor four, two message bits, two carry bits, and TUniform noise, also reported in Table 2; its P_{fail} entry is a per-PBS estimate.

4.2 Evaluation Key Size Comparison

Table 3 reports the evaluation-key payload of the parameter sets in Table 2, listing our CPU and GPU settings separately because grouping-factor-four multi-bit blind rotation changes the blind-rotation key. BR/PBS is the blind-rotation/bootstrapping key, KS is pre-PBS key switching, Auto. is the complete automorphism-key set, and SS is scheme switching. The GPU BR/PBS entries use the grouping-factor-four multi-bit key layout. The totals count public evaluation material only; secret keys and transient evaluation memory are excluded. Multi-bit blind rotation dominates the GPU totals of 455.213 MB for ours and 316.621 MB for TFHE-rs. The Scytale entry is measured from its published Set II parameters under the same accounting; its smaller $\ell_{ggs w}$ and n_{LWE} yield a 15% smaller total than ours, which trades against its $1.56\text{--}3.23\times$ slower CxC (Table 6).

² Lattice Estimator: GitHub commit [3e48ef4](#), latest upstream as of July 25, 2026.

Table 3. Evaluation-key payload sizes for the parameter sets in Table 2 (decimal MB).

Work	BR/PBS	KS	Auto.	SS	Total
Ours	113.246	0.459	1.475	0.295	115.474
Ours GPU	452.985	0.459	1.475	0.295	455.213
TFHE-rs/SMOOTHIE	60.162	30.114	–	–	90.276
TFHE-rs GPU	241.172	75.448	–	–	316.621
Head Start	50.790	88.998	–	–	139.788
Scytale [32]	94.372	0.688	2.458	0.295	97.812

Table 4. CxP online latency (s), using 64 CPU threads, with each method’s peak resident host memory in parentheses (decimal GB), measured per method in an isolated process and therefore including that method’s evaluation keys. HS denotes Head Start; speedup is the fastest supported baseline latency divided by Ours.

n	TFHE-rs [38]	SMOOTHIE [27]	HS [13]	Ours	Speedup
8	0.07 (0.13)	0.07 (0.13)	0.09 (0.15)	0.04 (0.24)	1.50
16	0.10 (0.14)	0.09 (0.14)	0.11 (0.16)	0.06 (0.24)	1.58
32	0.16 (0.13)	0.14 (0.14)	0.14 (0.17)	0.09 (0.24)	1.53
64	0.33 (0.14)	0.26 (0.14)	0.22 (0.21)	0.17 (0.27)	1.29
128	0.91 (0.23)	0.68 (0.24)	0.57 (0.33)	0.39 (0.41)	1.44
256	3.19 (0.54)	2.15 (0.63)	1.54 (0.79)	1.08 (0.68)	1.42
512	11.98 (1.78)	6.85 (1.69)	4.71 (2.27)	3.77 (1.19)	1.25

4.3 CxP and CxC Multiplication Results

We evaluate CxP and CxC at $n \in \{8, 16, 32, 64, 128, 256, 512\}$ bits using 64 CPU threads, and report them in Tables 4 and 5. CxP is compared with TFHE-rs, SMOOTHIE, and Head Start; CxC is compared with TFHE-rs alone, because the other two methods require a public multiplier.

Across the seven widths in Table 4, CxP is 1.52–3.18 $\times$ faster than TFHE-rs, 1.48–1.98 $\times$ faster than SMOOTHIE, and 1.25–2.10 $\times$ faster than Head Start. The advantage over TFHE-rs generally grows with n , while the closest comparison at large widths is Head Start. The compression at small n is a wave effect: each operation is a short chain of dependent blind-rotation waves whose latency is nearly width-independent, and below $n \approx 64$ one wave holds too few independent rotations to fill 64 threads, so part of the machine idles. The memory figures show the same inversion as the GPU comparison below: our footprint starts at a fixed 0.24 GB floor of keys and VP scratch while the baselines idle near 0.14 GB, but grows only 5.1 $\times$ through $n = 512$ against their 12.5–14.8 $\times$, ending as the smallest footprint at the largest width.

The Head Start CxP comparison uses a different format contract [13]. Its input is a radix ciphertext whose blocks use a six-bit extended message-carry space: two message bits and four carry bits, in addition to the padding bit. Its output uses the regular four-bit message-carry space: two message bits and two

Table 5. CxC online latency (s), using 64 CPU threads, with each method’s peak resident host memory in parentheses (decimal GB), measured per method in an isolated process and therefore including that method’s evaluation keys. Speedup is the TFHE-rs latency divided by Ours.

n	TFHE-rs [38]	Ours	Speedup
8	0.07 (0.14)	0.04 (0.24)	1.78
16	0.12 (0.13)	0.08 (0.24)	1.51
32	0.27 (0.13)	0.17 (0.27)	1.64
64	0.77 (0.14)	0.36 (0.35)	2.15
128	2.79 (0.24)	0.99 (0.48)	2.83
256	10.80 (0.65)	3.53 (0.75)	3.06
512	42.85 (2.28)	12.55 (1.34)	3.41

Table 6. Single-threaded CxC online latency (s) against the multiplier generated by the Scytale compiler [32] at its published $2^{-130.41}$ parameter set, on the same CPU model. Speedup is Scytale divided by Ours.

n	Scytale [32]	Ours	Speedup
8	0.22	0.141	1.56
16	1.04	0.437	2.38
32	4.06	1.536	2.64
64	17.26	5.351	3.23

carry bits, in addition to padding; the final propagation included in our timer empties the output carry fields. The Head Start paper assumes that conversion to the extended input scale is absorbed by the final PBS of an earlier operation, and our benchmark likewise excludes that conversion.

The Head Start ratios are conservative in our favour: retuning our pipeline to 2^{-64} would permit a substantially faster parameter set than the 2^{-128} one measured here. We do not assign an unmeasured speedup factor to that setting.

CxC is $1.51\text{--}3.41\times$ faster than TFHE-rs across the same widths, with the same memory crossover as CxP: 1.34 GB against 2.28 GB at $n = 512$. SMOOTHIE and Head Start accept a public scalar and do not provide a ciphertext–ciphertext multiplication, so no CxC speedup can be reported against them.

The Scytale compiler [32] lowers general programs to the same CBS and VP primitives and reports single-threaded ciphertext–ciphertext multiplication. Table 6 compares our CxC against its published multiplier at its $2^{-130.41}$ parameter target; our runs use one thread on the same CPU model, under our standard operation-wide 2^{-128} plans. CxC is $1.56\text{--}3.23\times$ faster, and the gap grows with n . Both systems pay the same refined-CBS primitive in the same library, so the difference is algorithmic (Section 5).

Table 7. CxP-DP online latency (s), using 64 CPU threads, with each method’s peak resident host memory in parentheses (decimal GB), measured per method in an isolated process and therefore including that method’s evaluation keys. HS denotes Head Start; speedup is the fastest supported baseline latency divided by Ours.

n/m	TFHE-rs [38]	SMOOTHIE [27]	HS [13]	Ours	Speedup
8/8	0.17 (0.14)	0.14 (0.13)	0.17 (0.16)	0.15 (0.24)	0.98
8/128	0.88 (0.17)	0.36 (0.18)	0.42 (0.24)	0.19 (0.24)	1.90
8/1024	5.82 (0.43)	1.54 (0.57)	1.24 (0.66)	0.34 (0.28)	3.69
16/8	0.28 (0.14)	0.19 (0.13)	0.21 (0.18)	0.22 (0.23)	0.87
16/128	2.70 (0.29)	0.94 (0.30)	0.77 (0.39)	0.31 (0.24)	2.46
16/1024	19.14 (0.80)	4.25 (1.26)	2.61 (1.46)	0.69 (0.43)	3.78
32/8	0.65 (0.15)	0.37 (0.17)	0.31 (0.22)	0.35 (0.24)	0.89
32/128	7.82 (0.58)	2.40 (0.64)	1.69 (0.82)	0.63 (0.27)	2.66
32/1024	58.25 (1.56)	12.68 (3.36)	7.19 (3.76)	1.93 (0.75)	3.72
64/8	1.65 (0.29)	0.99 (0.27)	0.79 (0.38)	0.47 (0.27)	1.69
64/128	23.68 (1.60)	7.26 (1.89)	4.23 (2.21)	1.27 (0.40)	3.33
64/1024	185.85 (3.66)	41.98 (11.27)	19.08 (12.11)	5.87 (1.44)	3.25

4.4 CxP-DP Operation Results

We evaluate CxP-DP at $n \in \{8, 16, 32, 64\}$ with vector sizes $m \in \{8, 128, 1024\}$ and 64 CPU threads; this grid is the one reported by SMOOTHIE and Head Start for their dot-products, so the comparison uses each method at the widths its authors published.

Across all twelve CxP-DP points, our method is 1.18–31.65 $\times$ faster than TFHE-rs. It beats SMOOTHIE and Head Start at ten points each. At every $m \in \{128, 1024\}$ point, it is 4.60–31.65 $\times$ faster than TFHE-rs, 1.90–7.15 $\times$ faster than SMOOTHIE, and 2.23–3.78 $\times$ faster than Head Start. The exceptions are small-vector cases: at $(n, m) = (8, 8)$ and $(16, 8)$, SMOOTHIE is 1.02 $\times$ and 1.15 $\times$ faster, respectively; at $(16, 8)$ and $(32, 8)$, Head Start is 1.05 $\times$ and 1.12 $\times$ faster. The memory figures realize the in-place accumulation argument directly: at $n = 64$ our peak grows only 5.4 $\times$ from $m = 8$ to $m = 1024$ while the baselines grow 12.5–42.4 $\times$, ending at 1.44 GB against 3.7–12.1 GB.

The Head Start format and failure-target qualifications above apply unchanged here, so these ratios are also conservative in our favour.

4.5 GPU Comparison

Neither SMOOTHIE nor Head Start provides a GPU implementation [13, 27], so TFHE-rs is the only available GPU baseline. We implement our CxP, CxC, and CxP-DP algorithms with CUDA and compare them with the TFHE-rs GPU backend. The measurements use one NVIDIA RTX 5090. Our CBS and vertical-packing kernels use the GPU implementation of [31]. Host-to-device transfer is additionally outside the timed region, and the timer ends after device synchronization. Table 8 reports online latency, speedup over TFHE-rs, and peak device-memory use in decimal GB.

Table 8. GPU online latency, speedup, and peak device-memory use. Speedup is TFHE-rs GPU divided by Ours; CxP-DP uses $m = 1024$. Each peak-memory figure is measured with that implementation alone on the device.

Op.	n	Latency (s)		Spd.	Memory (GB)	
		TFHE-rs	Ours		TFHE-rs	Ours
CxP	8	0.049	0.016	3.14	2.20	2.20
	16	0.060	0.023	2.67	2.20	3.17
	32	0.076	0.034	2.27	2.20	3.71
	64	0.169	0.074	2.28	2.20	4.75
	128	0.404	0.230	1.76	2.97	6.46
	256	1.385	0.744	1.86	8.88	9.92
	512	4.656	2.597	1.79	22.37	15.82
CxC	8	0.042	0.015	2.86	2.20	3.11
	16	0.064	0.040	1.59	2.20	3.17
	32	0.142	0.073	1.96	2.20	4.82
	64	0.409	0.177	2.32	2.57	5.66
	128	1.485	0.569	2.61	7.37	6.59
	256	4.815	2.139	2.25	19.82	7.97
	512	14.526	8.126	1.79	28.27	12.13
CxP-DP	8	42.712	1.368	31.22	5.56	3.17
	16	54.455	1.849	29.45	8.24	3.34
	32	76.297	5.989	12.74	13.51	3.84
	64	156.587	48.642	3.22	20.69	6.23

The GPU noise budget. The grouping-factor-four multi-bit blind rotation of Table 2 shortens the dependent chain from n_{LWE} steps to $n_{\text{LWE}}/4$ but processes each group of four key bits with $2^4 - 1 = 15$ external products rather than four, leaving $15/4 = 3.75$ times the output variance at the same level. Both parameter sets are subject to the same $P_{\text{fail}} \leq 2^{-128}$ budget, so the GPU planner starts with less margin per bootstrap and must recover it by reducing more often.

CxP-DP exposes this most sharply, since the admissible column bound decides how many terms may accumulate before a split, and a larger variance shrinks it. Through $n = 32$ a flat merge still satisfies the budget and latency grows smoothly, from 1.37 to 1.85 to 5.99 s. At $n = 64$ none does, the framework escalates to a hierarchical merge, and latency steps to 48.64 s: a factor of 8.1 against 3.2 from 16 to 32, which is why the speedup falls from 12.74 to 3.22. The step is a noise-budget effect, not a throughput one: the CPU parameter set admits the flat merge at $n = 64$, where the same column of Table 7 grows by only 3.04. Both GPU dot-products are slower than their CPU runs at these sizes because CxP-DP spends most of its time in bootstrap-free LWE additions and a few dependent reduction rounds, leaving device throughput largely unused; whether a GPU pays off is therefore application-dependent, and the GPU column compares the two implementations.

Speedup and memory across n . Our framework issues fewer blind rotations than the TFHE-rs integer pipeline, but each costs more, since a circuit bootstrap

Table 9. AlexNet layer multi-scalar multiplications (CxP-DP) at $n = 16$: online latency (s) with peak memory in parentheses (decimal GB). CPU rows use 64 threads and report peak resident host memory, measured per method in an isolated process including its evaluation keys; GPU rows report peak device memory with each implementation alone on the device. HS denotes Head Start; SMOOTHIE and Head Start provide no GPU implementation. Speedup is the fastest supported baseline latency divided by Ours.

	Layer	TFHE-rs [38]	SMOOTHIE [27]	HS [13]	Ours	Speedup
CPU	CONV2	45.70 (1.54)	8.81 (2.86)	5.52 (3.23)	1.21 (0.81)	4.56
	FC2	77.78 (2.46)	13.56 (4.47)	8.54 (4.78)	1.78 (1.27)	4.81
	FC1	175.02 (5.38)	28.65 (9.83)	16.28 (10.03)	3.64 (2.65)	4.47
GPU	CONV2	94.41 (12.57)	–	–	3.76 (3.54)	25.10
	FC2	161.19 (19.25)	–	–	34.70 (4.21)	4.65
	FC1	545.93 (22.30)	–	–	78.68 (6.03)	6.94

adds ℓ_{ggsu} HomTrace operations and a scheme switch that a classic bootstrap never pays. At small n neither implementation fills the device and latency tracks dependent waves, so the count advantage transfers almost intact; as n grows both saturate the device and the cheaper TFHE-rs blind rotation recovers part of the gap. Peak memory follows the same public plan that fixes the operation counts, so it is computable rather than only measurable. Its dominant terms are the evaluation keys and one selector set, created once per operand and reused by every VP evaluation; the reduction then consumes each column layer as it is produced instead of retaining every intermediate. This is why the memory column of Table 8 inverts: vertical packing carries a fixed cost the baseline has no counterpart for at small n , while at large n the baseline holds a full set of intermediate radix ciphertexts. CxP-DP creates no selectors and accumulates in place, so its footprint is nearly independent of m beyond the inputs themselves and stays below the baseline at every width.

4.6 AlexNet Layer Benchmarks

To evaluate CxP-DP on a neural-network workload, we adopt the AlexNet layer grid that SMOOTHIE reports for privacy-preserving inference. Each benchmark is the dot product behind one output activation of its layer, a 16-bit multi-scalar multiplication of public weights with encrypted inputs: CONV2 convolves a 5×5 window over its 96 input channels, so one output activation is a $5 \times 5 \times 96 = 2400$ -element dot product with the encrypted input patch; FC1 maps the flattened $6 \times 6 \times 256 = 9216$ feature map onto one neuron of the first fully connected layer; and FC2 connects the two 4096-neuron fully connected layers, one 4096-element dot product per neuron. A complete layer repeats one such product per output activation over the same encrypted inputs. SMOOTHIE and Head Start run SMOOTHIE’s published window per layer (7, 9, and 8) with bucket doubling. Its published figures use 16 threads and omit the final carry propagation, so all three methods are re-measured here under the protocol of this section: 64

threads, 16-bit scalars and encrypted values drawn from a fixed pseudorandom pattern and identical for every method, and a carry-resolved output.

Table 9 reports the results. On the CPU, Head Start is the fastest baseline at every layer; CxP-DP is $4.47\text{--}4.81\times$ faster than it, $7.28\text{--}7.87\times$ faster than SMOOTHIE, and $37.7\text{--}48.1\times$ faster than TFHE-rs, with the smallest memory footprint at every layer, 2.65 GB against 9.83–10.03 GB at FC1. The Head Start figures also carry a composability cost these single-kernel timings do not show: its outputs use the regular message-carry encoding while its inputs require the extended one, so a network chaining these layers pays one encoding conversion per layer, which neither our closed CC2 format nor the radix-to-radix TFHE-rs and SMOOTHIE pipelines incur. On the GPU, only TFHE-rs offers a comparison: CxP-DP is $4.65\text{--}25.10\times$ faster at 3.54–6.03 GB of device memory against 12.57–22.30 GB. The GPU noise budget behaves as in Table 8: the flat merge is admissible at CONV2, while FC2 and FC1 exceed the flat budget and escalate to hierarchical merging with smaller groups, so their GPU latencies trail their own CPU runs. Every accepted plan clears the 2^{-128} budget, at 2^{-179} on the CPU, and the layer inputs arrive and the products leave in closed CC2, so a full linear layer composes these calls directly.

5 Related Work

TFHE-rs [1, 38] is the main baseline for TFHE integer arithmetic. It evaluates small block products with bivariate PBS, reduces partial-sum columns, and propagates carries to restore its message-carry radix form. In contrast, our CC2 boundary reserves nine carry bits to accumulate bounded columns without bootstrapping, uses CBS and VP for product generation, and defers carry propagation to a single final carry-select stage.

The original TFHE TBSR counter [8, 9] provides a leveled multiplier utilizing column-oriented schoolbook multiplication with CMux and functional key switching. While TBSR maintains column height in an encrypted counter (requiring serial CMux evaluations and frequent carry propagations) our method represents column height as a public bound. This enables free LWE accumulation and defers the entire carry chain to a single terminal, allowing large dot-products to form their columns without any bootstrapping.

Trifan et al. [34] accelerate bitwise TFHE multiplication using parallel carry-save updates and ternary gates, but explicitly avoid Dadda trees due to high gate-count assumptions. Our framework overcomes this limitation by extracting both compressor outputs from a single blind rotation, making Dadda reduction highly efficient on standard CPUs. While they report performance improvements over older TFHE parameters, a direct empirical comparison is not feasible as their implementation is not public and targets weaker security constraints.

Scytale [32] is a general-purpose compiler framework that maps arbitrary programs into batched CBS and VP operations. Because it relies on gate-level synthesis, its generated multiplier materializes all partial products, adders, and carries as expensive LUT evaluations, requiring repeated circuit-bootstrapping

between layers. Our algorithms replace this generated structure with optimized arithmetic: operand selectors are reused, public column bounds enable free accumulation, and the carry chain is resolved exactly once. As shown in Table 6, this algorithmic specialization makes our CxC $1.56\text{--}3.23\times$ faster than Scytale’s generated multiplier.

Recent works optimize TFHE scalar multiplication and vector addition using standard message-carry radix arithmetic. SMOOTHIE [27] adapts the bucket method from elliptic-curve cryptography for multi-scalar multiplication, introducing techniques like power-shift aggregation to reduce bootstraps. Head Start [13] builds upon SMOOTHIE by introducing an MSB-to-LSB digit-decomposition method (*DirtyMSB*) that allows accumulating more radix ciphertexts in an enlarged plaintext space before decomposition. While these works optimize digit extraction and carry normalization within radix pipelines, our framework fundamentally alters how products are generated and reduced via CBS and VP, and uniquely supports ciphertext–ciphertext multiplication.

6 Concluding Remarks

We presented a unified TFHE framework for ciphertext–plaintext (CxP) multiplication, ciphertext–ciphertext (CxC) multiplication, and ciphertext–plaintext dot-products (CxP-DP). All three operations compose seamlessly without conversion using our CC2 encoding, which provides inherent carry capacity for bootstrap-free accumulation. By utilizing a shared weighted-column representation and a fused CBS/CMux carry-select terminal, our framework substantially accelerates exact-integer arithmetic.

Our evaluations demonstrate significant performance gains over TFHE-rs on both a 64-thread CPU (up to $31.65\times$ faster) and a GPU (up to $31.22\times$ faster). Furthermore, our framework outperforms recent baselines like SMOOTHIE and Head Start across most workloads, including AlexNet layers, where CxP-DP runs up to $4.81\times$ faster than the fastest baseline while requiring nearly a quarter of the peak memory. Ultimately, we demonstrate that circuit bootstrapping and vertical packing, long thought too slow for arithmetic, now practically and efficiently outperform standard PBS-based integer pipelines at every evaluated width.

Acknowledgments

This work has been supported by NSF Award #2239334.

References

1. Mathieu Ballandras, Loris Bergerat, Charlotte Bonte, Carl Bootland, Benjamin R. Curtis, Jad Khatib, Jakub Klemsa, Thomas Montaigu, Jean-Baptiste Orfila, and Samuel Tap. TFHE-rs: A (Practical) Handbook, 2026. First edition, version 1.6. <https://github.com/zama-ai/tfhe-rs-handbook/blob/main/tfhe-rs-handbook.pdf>.

2. Loris Bergerat, Anas Boudi, Quentin Bourgerie, Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. Parameter optimization and larger precision for (T)FHE. *Journal of Cryptology*, 36(3):28, 2023.
3. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.
4. Sergiu Carpov, Malika Izabachène, and Victor Mollimard. New techniques for multi-value input homomorphic evaluation and applications. In *Cryptographers’ Track at the RSA Conference*, pages 106–126. Springer, 2019.
5. Yangyi Chen, Bo Peng, XiaoFeng Wang, and Haixu Tang. Large-scale privacy-preserving mapping of human genomic sequences on hybrid clouds. In *NDSS*, 2012.
6. Jung Hee Cheon, Hyeongmin Choe, Alain Passelègue, Damien Stehlé, and Elias Suvanto. Attacks Against the IND-CPA^D Security of Exact FHE Schemes. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 2505–2519, 2024.
7. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*, pages 409–437. Springer, 2017.
8. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster Packed Homomorphic Operations and Efficient Circuit Bootstrapping for TFHE. In *ASIACRYPT*, volume 10624 of *Lecture Notes in Computer Science*, pages 377–408. Springer, 2017.
9. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast Fully Homomorphic Encryption over the Torus. *Journal of Cryptology*, 33(1):34–91, 2020.
10. Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE. In *ASIACRYPT*, pages 670–699. Springer, 2021.
11. Luigi Dadda. Some schemes for parallel multipliers. *Alta Frequenza*, 34:349–356, 1965.
12. Joan Daemen and Vincent Rijmen. AES proposal: Rijndael, 1999.
13. Jan-Pieter D’Anvers, Xander Pottier, Thomas de Ruijter, and Ingrid Verbauwhede. Head Start: Digit Extraction in TFHE from MSB to LSB. Cryptology ePrint Archive, Paper 2025/2012, 2025. <https://eprint.iacr.org/2025/2012>.
14. Thomas de Ruijter, Jan-Pieter D’Anvers, and Ingrid Verbauwhede. Don’t be mean: Reducing approximation noise in TFHE through mean compensation. Cryptology ePrint Archive, Report 2025/809, 2025.
15. Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In *EUROCRYPT*, pages 617–640. Springer, 2015.
16. Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012.
17. Craig Gentry. Fully homomorphic encryption using ideal lattices. In *ACM Symposium on Theory of Computing*, pages 169–178, New York, NY, USA, 2009. Association for Computing Machinery.
18. Tahseen Khan, Wenhong Tian, Guangyao Zhou, Shashikant Ilager, Mingming Gong, and Rajkumar Buyya. Machine learning (ML)-centric resource management in cloud computing: A review and future directions. *Journal of Network and Computer Applications*, 204:103405, 2022.
19. Ben Langmead and Abhinav Nellore. Cloud computing for genomic data analysis and collaboration. *Nature Reviews Genetics*, 19(4):208–219, 2018.

20. Kang Hoon Lee and Ji Won Yoon. Homomorphic field trace revisited: Breaking the cubic noise barrier. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2026(1):105–134, 2026.
21. Baiyu Li and Daniele Micciancio. On the Security of Homomorphic Encryption on Approximate Numbers. In *EUROCRYPT*, volume 12696 of *Lecture Notes in Computer Science*, pages 648–677. Springer, 2021.
22. Bo Liu, Ravi K Madduri, Borja Sotomayor, Kyle Chard, Lukasz Lacinski, Utpal J Dave, Jianqiang Li, Chunchen Liu, and Ian T Foster. Cloud-based bioinformatics workflow platform for large-scale next-generation sequencing analyses. *Journal of biomedical informatics*, 49:119–133, 2014.
23. Álvaro López García, Jesús Marco de Lucas, Marica Antonacci, Wolfgang zu Castell, Mário David, Marcus Hardt, Lara Lloret Iglesias, Germán Moltó, Marcin Plóciennik, Viet Tran, Andy S. Alic, Miguel Caballer, Isabel Campos Plasencia, Alessandro Costantini, Stefan Dlugolinsky, Doina Cristina Duma, Giacinto Donvito, Jorge Gomes, Ignacio Heredia Cacha, Keiichi Ito, Valentin Y. Kozlov, Giang Nguyen, Pablo Orviz Fernández, Zdeněk Šustr, and Pawel Wolniewicz. A cloud-based framework for machine learning workloads and applications. *IEEE Access*, 8:18681–18692, 2020.
24. Wen-jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. PEGASUS: Bridging Polynomial and Non-polynomial Evaluations in Homomorphic Encryption. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1057–1073, 2021.
25. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *EUROCRYPT*, pages 1–23. Springer, 2010.
26. Luis Montero, Jordan Frery, Celia Kherfallah, Roman Bredehoft, and Andrei Stoian. Machine learning training on encrypted data with tfhe. In *Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics*, pages 71–76, 2024.
27. Xander Pottier, Jan-Pieter D’Anvers, Thomas de Ruijter, and Ingrid Verbauwhede. SMOOTHIE: (Multi-)Scalar Multiplication Optimizations On TFHE. *Cryptology ePrint Archive, Paper 2025/1267*, 2025.
28. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):1–40, 2009.
29. Mauro Ribeiro, Katarina Grolinger, and Miriam AM Capretz. Mlaas: Machine learning as a service. In *2015 IEEE 14th international conference on machine learning and applications (ICMLA)*, pages 896–902. IEEE, 2015.
30. Rostin Shokri and Nektarios Georgios Tsoutsos. CHESS: Compiling Homomorphic Encryption with Scheme Switching. In *2025 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 324–334, 2025.
31. Rostin Shokri and Nektarios Georgios Tsoutsos. Seamless Switching Between PBS and WoPBS for Scalable TFHE. *Cryptology ePrint Archive, Paper 2025/777*, 2025. <https://eprint.iacr.org/2025/777>.
32. Rostin Shokri and Nektarios Georgios Tsoutsos. Scytale: A Compiler Framework for Accelerating TFHE with Circuit Bootstrapping. In *2026 Design, Automation & Test in Europe Conference (DATE)*, 2026.
33. Andrei Stoian, Jordan Frery, Roman Bredehoft, Luis Montero, Celia Kherfallah, and Benoit Chevallier-Mames. Deep neural networks for encrypted inference with tfhe. In *International Symposium on Cyber Security, Cryptology, and Machine Learning*, pages 493–500. Springer, 2023.

34. Marc Titus Trifan, Alexandru Nicolau, and Alexander V. Veidenbaum. Enhancing the Privacy of Machine Learning via Faster Arithmetic over Torus FHE. In *IEEE International Conference on Cyber Security and Cloud Computing (CSCloud)*, pages 46–52. IEEE, 2023.
35. Ruida Wang, Jincheol Ha, Xuan Shen, Xianhui Lu, Chunling Chen, Kunpeng Wang, and Jooyoung Lee. Refined TFHE leveled homomorphic evaluation and its application. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 2309–2323. Association for Computing Machinery, 2025.
36. Ruida Wang, Yundi Wen, Zhihao Li, Xianhui Lu, Benqiang Wei, Kun Liu, and Kunpeng Wang. Circuit bootstrapping: faster and smaller. In *EUROCRYPT*, pages 342–372. Springer, 2024.
37. Stephen S Yau and Ho G An. Protection of users’ data confidentiality in cloud computing. In *Proceedings of the Second Asia-Pacific Symposium on Internetware*, pages 1–6, 2010.
38. Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. <https://github.com/zama-ai/tfhe-rs>.
39. Yinqian Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Cross-VM side channels and their use to extract private keys. In *ACM Computer and Communications Security*, pages 305–316, 2012.
40. Yinqian Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Cross-tenant side-channel attacks in PaaS clouds. In *ACM Computer and Communications Security*, pages 990–1003, 2014.